

CS486 SOLUTION - TEST 1

Based on the FINAL REVIEW structure; total: 10 points

Question 1 - ER Models

[1 pt]

Solution. Use entities **Club**(ClubName), **Team**(ClubName,TeamName), **Athlete**(ClubName,TeamName,JerseyNo) and **Coach**(CoachID,...). ClubName is the key of Club. Team is existence-dependent on Club, with composite key (ClubName,TeamName). Athlete is existence-dependent on Team, with composite key (ClubName,TeamName,JerseyNo).

Relationships: Club 1:N Team with total participation of Team; Team 1:N Athlete with total participation of Athlete; Coach M:N Team through Coaches; and a role/constraint HeadCoach such that each Team has exactly one head coach and that head coach must also participate in Coaches for that Team. A relational implementation can use Team.HeadCoachID as an FK to Coach plus a constraint ensuring membership in TeamCoach.

Question 2 - Query Languages

[2 pt]

Solution. (a) Let $D = \sigma_{CID='DB'}(Prerequisite)$. Students who have all prerequisites can be expressed by division:

$$\pi_{SID,CID}(Enroll) \div \pi_{PreCID}(D)$$

after renaming the enrolled CID to PreCID and projecting SID from the division result. More explicitly,

$$\rho_{SID,PreCID}(\pi_{SID,CID}(Enroll)) \div \pi_{PreCID}(\sigma_{CID='DB'}Prerequisite).$$

(b)

$\{s \mid Student(s) \wedge \exists e \exists c (Enroll(e) \wedge Course(c) \wedge e.SID = s.SID \wedge e.CID = c.CID \wedge c.Dept = s.Major)\}$.

(c) One window-function solution:

```
WITH cnt AS (
  SELECT s.SID, s.Name, s.Major,
         COUNT(DISTINCT CASE WHEN e.Grade >= 5 THEN e.CID END) AS passed
  FROM Student s LEFT JOIN Enroll e ON e.SID=s.SID
  GROUP BY s.SID,s.Name,s.Major
), r AS (
  SELECT *, DENSE_RANK() OVER(PARTITION BY Major ORDER BY passed DESC) AS rk
  FROM cnt
)
SELECT Major,SID,Name,passed FROM r WHERE rk=1;
```

Question 3 - Database Constraints

[1 pt]

Solution. Formal forbidden-state TRC:

$\neg \exists e \exists p \exists w (Employee(e) \wedge Project(p) \wedge WorksOn(w) \wedge w.EID = e.EID \wedge w.PID = p.PID \wedge e.Dept \neq p.Dept)$.

Row-local rule: CHECK (Hours > 0 AND Hours <= 40). Foreign keys enforce EID/PID existence. The same-department rule is cross-table and typically needs an assertion (if supported) or an INSERT/UPDATE trigger that joins **inserted** to Employee and Project and rejects any row where departments differ.

Question 4 - Serialization & Concurrency**[2 pt]**

Solution. Conflicts give edges: on A, $T_1 \rightarrow T_2$, $T_1 \rightarrow T_3$, and $T_2 \rightarrow T_3$; on B, $T_3 \rightarrow T_1$ and $T_1 \rightarrow T_2$ (also $T_3 \rightarrow T_2$ through $r_3(B)$ before $r_2(B)$ is read-read, so no edge from that pair); on C, $T_2 \rightarrow T_1$. There is a cycle $T_1 \rightarrow T_3 \rightarrow T_1$ and also $T_1 \rightarrow T_2 \rightarrow T_1$. Hence it is not conflict-serializable.

For recoverability, the relevant reads are $r_2(B)$ after $w_1(B)$, so T_2 depends on T_1 ; $r_1(C)$ after $w_2(C)$, so T_1 depends on T_2 . Commit order is c_3, c_1, c_2 : T_1 commits before T_2 despite reading from T_2 , so the schedule is not recoverable. It is not cascadeless because there are reads of uncommitted values.

Strict 2PL would not permit the exact interleaving because exclusive locks are held until commit/abort; the shown cross-dependencies would force blocking rather than immediate conflicting access.

Question 5 - Recovery**[1 pt]**

Solution. Committed winner: T1. T2 explicitly aborted before crash, so its effects should already have been undone (or must be completed if rollback was not complete). T3 is an uncommitted loser.

REDO committed T1 as needed: $A := 14$. If the system uses repeating-history style recovery, it may redo logged updates then undo losers; conceptually the final database must contain T1 and exclude T2/T3. UNDO T3 in reverse order: restore C from 38 to 36, then from 36 to 30. Ensure T2's D is restored 44 to 40 and B 25 to 20 if its abort rollback had not completed before the crash.

Question 6 - Functional Dependencies & Normalization**[1 pt]**

Solution. $A^+ = \{A, B, C, D, E, F\}$ because $A \rightarrow BC$, $C \rightarrow D$, $D \rightarrow E$, then $BE \rightarrow F$. Therefore A is a candidate key (single attribute, hence minimal).

$C \rightarrow D$ and $D \rightarrow E$ have non-superkey determinants and nonprime RHS attributes, so 3NF is violated. With a single-attribute key A, there is no partial dependency, so the relation is in 2NF but not 3NF (therefore not BCNF).

The proposed three-way decomposition is not guaranteed lossless from the listed FDs. In particular, $R_1 \cap R_2 = \{C\}$ and $C \rightarrow R_2$ holds for CDE , but joining $R_3(B, E, F)$ through BE requires an interaction not ensured to connect without spurious tuples in all legal instances. A chase/common-attribute proof cannot establish the whole decomposition as lossless.

Question 7 - Concurrency Control Execution**[1 pt]**

Solution. Serial T1 then T2: $100 \rightarrow 80 \rightarrow 88$. Serial T2 then T1: $100 \rightarrow 110 \rightarrow 90$. Thus serial results are 88 and 90.

A lost-update interleaving can give 80: both read 100; T2 computes 110, T1 computes 80; T2 writes 110 then T1 overwrites with 80. It can also give 110 if T1 writes 80 then T2 overwrites with 110. Therefore distinct final values are $\{80, 88, 90, 110\}$. Values 80 and 110 are lost-update anomalies.

Question 8 - Query Optimization**[1 pt]**

Solution. OrderID equality: B+ tree about 3 index-page I/Os plus 1 data-page I/O ≈ 4 ; hash about 2 bucket/index I/Os plus roughly 1 data-page I/O ≈ 3 (depending on whether the bucket stores the record/RID). Hash is slightly better for pure equality.

For one Status value, uniformity gives 100,000 tuples = about 1,000 data pages if perfectly clustered, but the index is unclustered; in the worst/common case many qualifying tuples are scattered across up to roughly 10,000 pages. Index descent is negligible compared with random fetches. A full scan costs 10,000 sequential page I/Os and is likely preferable to an unclustered low-selectivity index. Assumptions: no covering index; one page I/O per fetched data page; uniform distribution.